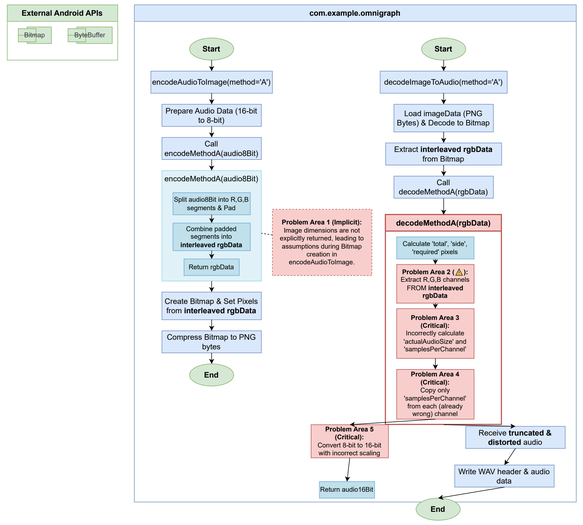
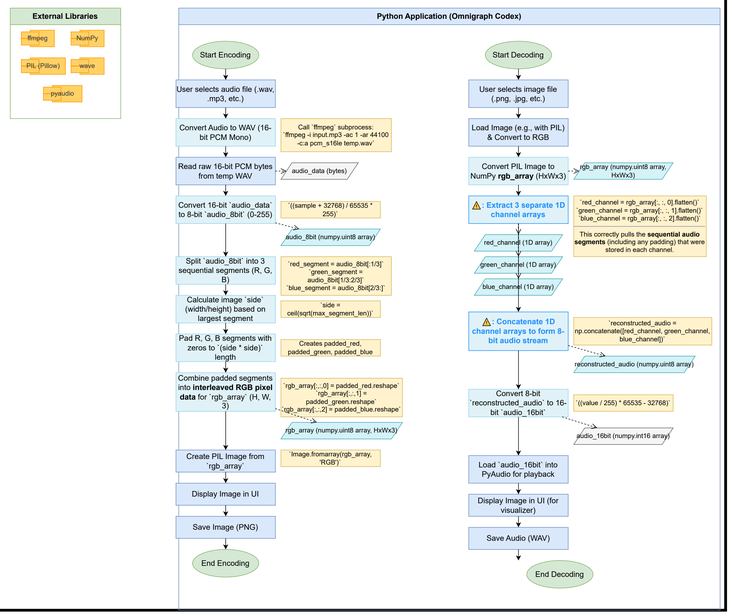
Kotlin VS PYTHON

Kotlin – Method A (Encoding & Decoding)

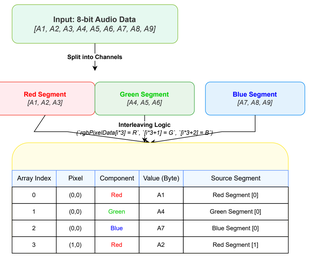


Correct Python Logic Flow – Method A (Encoding & Decoding)



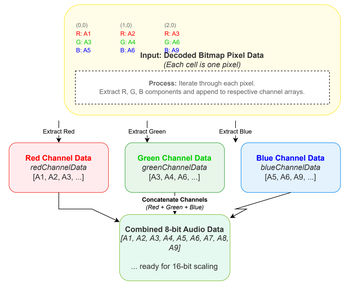
PIXEL AND CHANNEL STUFF

ENCODING METHOD A: PIXEL CONSTRUCTION DETAIL



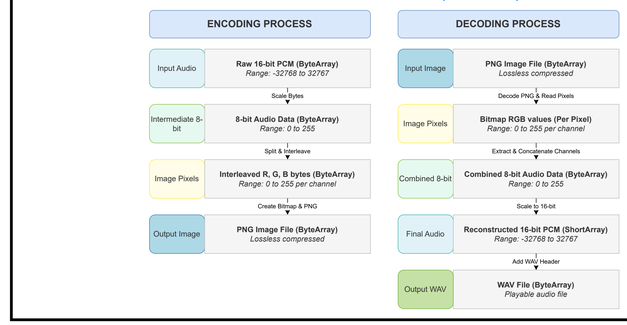
Array Index	Pixel	Component	Value (Byte)	Source Segment
0	(0,0)	Red	A1	Red Segment [0]
1	(0,0)	Green	A4	Green Segment [0]
2	(0,0)	Blue	A7	Blue Segment [0]
3	(1,0)	Red	A2	Red Segment [1]

DECODING METHOD A: CHANNEL EXTRACTION DETAIL



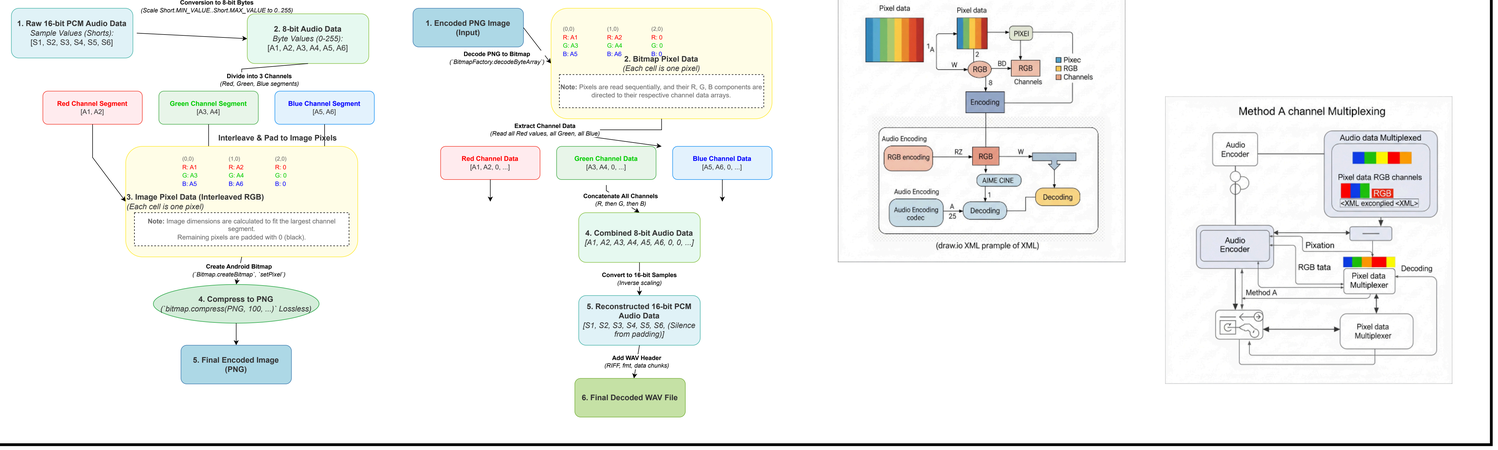
DFF Method A

DATA TRANSFORMATION FLOW (METHOD A)



Method A Channel Multiplexing : The Idea

AUDIO ENCODING & DECODING: METHOD A (CHANNEL MULTIPLEXING)




```
/* ********************************************************************************************
 * ENCODING METHOD IMPLEMENTATIONS
 *
 * These private methods implement the specific logic for each encoding scheme.
 ******************************************************************************************** */

/**
 * Implements encoding Method A (Channel Multiplexing).
 * Stores audio sequentially in separate color channels (Red -> Green -> Blue).
 *
 * @param audio8Bit The 8-bit audio data (pixel intensity values).
 * @return A Triple containing:
 * 1. A ByteArray of interleaved R,G,B pixel data ready for Bitmap creation.
 * 2. The calculated width of the resulting square image.
 * 3. The calculated height of the resulting square image.
 */
private fun encodeMethodA(audio8Bit: ByteArray): Triple<ByteArray, Int, Int> {
    val totalAudioLength = audio8Bit.size

    // Calculate sizes for each channel, splitting the total audio length as evenly as possible.
    // Any remainder from integer division goes to the blue channel.
    val redSize = totalAudioLength / 3
    val greenSize = totalAudioLength / 3
    val blueSize = totalAudioLength - redSize - greenSize

    // Create separate segments for Red, Green, and Blue channels.
    val redSegment = audio8Bit.copyOfRange(0, redSize)
    val greenSegment = audio8Bit.copyOfRange(redSize, redSize + greenSize)
    val blueSegment = audio8Bit.copyOfRange(redSize + greenSize, totalAudioLength)

    // Determine the image dimensions based on the longest audio segment.
    // This ensures all audio data (including padding) will fit into the square image.
    val maxSegmentLength = maxOf(redSegment.size, greenSegment.size, blueSegment.size)
    val side = ceil(sqrt(maxSegmentLength.toDouble())).toInt() // Calculate side of the square image.
    val requiredPixelsPerChannel = side * side // Total pixels needed for one channel's data (accounting for square padding).

    Log.d("AudioProcessor", "Method A Encode Logic: Red/Green/Blue segment lengths: ${redSegment.size}/${greenSegment.size}/${blueSegment.size}. " +
            "Max segment length: $maxSegmentLength. Calculated image side: $side. Required pixels per channel: $requiredPixelsPerChannel.")

    // Create a single ByteArray to hold the final interleaved R, G, B pixel data for the image.
    // The total size is (required pixels per channel * 3 color channels).
    val rgbPixelData = ByteArray(requiredPixelsPerChannel * 3)

    // Populate the rgbPixelData array. Each pixel's R, G, B components are taken sequentially
    // from the corresponding Red, Green, and Blue audio segments. Padding with 0 if segment ends early.
    for (i in 0 until requiredPixelsPerChannel) {
        rgbPixelData[i * 3] = if (i < redSegment.size) redSegment[i] else 0.toByte() // Red channel gets data from redSegment.
        rgbPixelData[i * 3 + 1] = if (i < greenSegment.size) greenSegment[i] else 0.toByte() // Green channel gets data from greenSegment.
        rgbPixelData[i * 3 + 2] = if (i < blueSegment.size) blueSegment[i] else 0.toByte() // Blue channel gets data from blueSegment.
    }

    // Return the generated interleaved RGB pixel data along with the explicit width and height.
    return Triple(rgbPixelData, side, side)
}
```

```
/* ********************************************************************************************
 * DECODING METHOD IMPLEMENTATIONS
 *
 * These private methods implement the specific logic for each decoding scheme.
 ******************************************************************************************** */

/**
 * Implements decoding Method A (Channel Multiplexing).
 * Reconstructs audio by concatenating data extracted from the Red, Green, and Blue channels sequentially.
 *
 * @param redData The 8-bit data extracted from the Red channel of the image (first audio segment).
 * @param greenData The 8-bit data extracted from the Green channel of the image (second audio segment).
 * @param blueData The 8-bit data extracted from the Blue channel of the image (third audio segment).
 * @return A ShortArray representing the reconstructed 16-bit audio samples.
 */
private fun decodeMethodA(redData: ByteArray, greenData: ByteArray, blueData: ByteArray): ShortArray {
    Log.d("AudioProcessor", "Method A Decode Logic: Red data size: ${redData.size}, Green data size: ${greenData.size}, Blue data size: ${blueData.size}.")

    // Concatenate the three separate 8-bit channel data arrays into a single combined 8-bit audio stream.
    // This directly reverses the sequential splitting and storage done during encoding.
    val totalAudio8BitSize = redData.size + greenData.size + blueData.size
    val audio8BitCombined = ByteArray(totalAudio8BitSize)

    // Efficiently copy each channel's data into the combined array at the correct starting offset.
    redData.copyInto(audio8BitCombined, 0)
    greenData.copyInto(audio8BitCombined, redData.size)
    blueData.copyInto(audio8BitCombined, redData.size + greenData.size)

    Log.d("AudioProcessor", "Method A Decode Logic: Combined 8-bit audio stream size: ${audio8BitCombined.size}.")

    // Convert the combined 8-bit pixel intensity values (range 0-255) back to 16-bit audio samples
    // (range Short.MIN_VALUE to Short.MAX_VALUE). This is the mathematically correct inverse scaling.
    val audio16BitShorts = audio8BitCombined.map { b ->
        val value = b.toInt() and 0xFF // Convert signed byte to unsigned integer (0-255).
        // Inverse scaling: ((value / 255.0) * (range of short)) + min_short_value.
        ((value.toFloat() / 255.0f * (Short.MAX_VALUE - Short.MIN_VALUE)) + Short.MIN_VALUE).toInt().toShort()
    }.toShortArray()

    Log.d("AudioProcessor", "Method A Decode Logic: Final 16-bit audio samples reconstructed: ${audio16BitShorts.size}.")
    return audio16BitShorts
}
```

Passage : AUDIO >> IMAGE

OMNIGRAPH: AUDIO-IMAGE TRANSFORMATION (METHOD A)